

USE REF() HOOK - COMPLETE NOTES

Page: _____

1. What is useRef() ?

- `useRef()` is a React Hook that returns a mutable ref object.
- It allows you to persist values between renders without causing re-renders.
- Useful for accessing DOM elements, storing mutable values, timers, previous values, etc.

Mutable value
container
that survives
re-renders



2. Why use useRef() ?

- ✓ Access DOM elements directly.
- ✓ Store mutable values (does not cause re-render).
- ✓ Keep track of previous values.
- ✓ Manage timers, intervals, timeouts.
- ✓ Avoid unnecessary re-renders.

`useRef()`
= Persistent
+ Mutable
+ No Re-render



3. Basic Syntax

```
import { useRef } from 'react';

const myRef = useRef(initialValue);

// myRef is an object
// myRef.current holds the value
```



```
myRef
{
  current: initialValue
}
```

4. Important Points

- `useRef()` returns the same object on every render.
- Changing `ref.current` does NOT trigger a re-render.
- The `.current` property is mutable.
- The initial value is used only on the first render.

Same ref
object across
all renders



5. Accessing DOM Elements

```
import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const handleClick = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Type..." />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
}
```

Most common
use of `useRef`
→ Access DOM
directly



How it works ?

`useRef(null)`



ref object
{ current: null }



attached to
DOM element



`ref.current`
points to DOM

6. Storing Mutable Values (No Re-render)

```
import { useRef } from 'react';
function ClickCounter() {
  const countRef = useRef(0); // does not cause re-render

  const handleClick = () => {
    countRef.current = countRef.current + 1;
    console.log("Count:", countRef.current);
  };

  return <button onClick={handleClick}>Click Me </button>;
}
```

Changing
ref.current
does not
re-render
component



7. Previous Value Example

```
import { useRef, useState, useEffect } from 'react';
function ShowPreviousValue() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef();

  useEffect(() => {
    prevCountRef.current = count; // store previous value
  }, [count]);

  return (
    <div>
      <p>Current Count: {count}</p>
      <p>Previous Count: {prevCountRef.current}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

useRef helps
store previous
value without
extra re-render



8. useRef with setTimeout / setInterval

```
function Timer() {
  const timerRef = useRef(null);

  const startTimer = () => {
    timerRef.current = setTimeout(() => {
      alert("Time is up!");
    }, 3000);
  };

  const clearTimer = () => {
    clearTimeout(timerRef.current);
  };

  return (
    <div>
      <button onClick={startTimer}>Start </button>
      <button onClick={clearTimer}>Cancel </button>
    </div>
  );
}
```

Store timer/id
in ref so we
can clear it
later



9. Common useRef() Use Cases

- Focus management
- DOM measurements
- Store timers / intervals
- Previous values
- Persist values without re-render
- Scroll position
- Video / Audio control

Many real-life
scenarios in
React apps



10. Difference: useRef vs useState

<u>Feature</u>	<u>useState</u>	<u>useRef</u>
Purpose	Manage state	Stores mutable values / DOM reference
Triggers Re-render	Yes	No
Update	Using <code>setState()</code>	Directly change <code>ref.current</code>
Syntax	<code>const [state, setState] = useState(initial)</code>	<code>const ref = useRef(initial)</code>
Use Cases	UI data that affects render	DOM access, timers, previous values, etc.

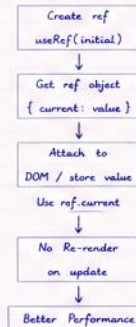
11. Important Notes

- ✓ Do not use `useRef` for data that affects UI.
- ✓ Do not read or write `ref.current` during rendering (except initial use).
- ✓ Use `ref` with `forwardRef` to pass refs to child components.
- ✓ `useRef` is perfect for performance optimizations.

Use smartly
for better
performance

12. Quick Revision

- `useRef()` returns a `ref` object with `.current` property.
- `.current` is mutable and does not cause re-render.
- Use `ref` for DOM access, timers, previous values.
- Use `useState` for UI-related state.
- Same `ref` object persists across renders.

Complete Flow13. Mini Example Summary

```

const inputRef = useRef(null);
<input ref={inputRef} />

inputRef.current.focus(); // access DOM
inputRef.current = 10; // store value
inputRef.current = "New"; // update without re-render
  
```

One Hook
Many
Possibilities



★ `useRef()` = Mutable + Persistent + No Re-render = Power Tool ★